Topic 1: Logistic Regression

0.1 Introduction: What Is Classification?

Imagine you receive an email. Before you even open it, your email service has already made a decision: is this message *spam* or *not spam*? Somewhere behind the scenes, a computer program looked at the email and sorted it into one of two categories. That act of sorting things into categories is called **classification**, and it is one of the most widely used ideas in machine learning.

Definition: Classification

Classification is a type of **supervised learning** in which a model learns from labelled examples in order to predict a **category** (also called a *class* or *label*) for new, unseen data. The output is always a discrete category, such as "spam / not spam", "disease / no disease", or "cat / dog / bird".

0.1.1 Supervised Learning in Plain Words

The word *supervised* means that we learn from examples where the correct answer is already known. Think of a young child learning to recognise fruit. A parent shows the child an apple and says "apple", then shows a banana and says "banana". After seeing many labelled examples, the child can identify a fruit they have never seen before. In machine learning terms:

- The fruits shown to the child are the **training data**.
- The names spoken by the parent are the **labels**.
- The child's growing ability to recognise fruit is the **model**.
- Identifying a brand-new fruit is **prediction** (or *inference*).

0.1.2 Classification vs. Regression

Supervised learning problems come in two flavours, and it is important to tell them apart from day one.

Table 1: Classification versus regression at a glance.

	Classification	**Regression**
Output type	A category (discrete)	A number (continuous)
Example question	"Is this email spam?"	"What will this house sell for?"
Example answers	Yes / No	\$312,500
Typical evaluation	Accuracy, precision, recall	Mean squared error

A simple test: if the answer to your question is a *name* or a *yes/no*, it is classification. If the answer is a *quantity*, it is regression.

0.1.3 Some Vocabulary We Will Use Throughout

- **Feature** (x): an input measurement, e.g. the number of exclamation marks in an email, a patient's age, a fruit's weight.
- **Label** (y): the correct answer for a training example, e.g. "spam".
- **Training set**: the labelled examples the model learns from.
- **Test set**: labelled examples kept aside to check how well the model performs on data it has never seen.
- **Binary classification**: exactly two classes (spam / not spam).
- **Multiclass classification**: three or more classes (cat / dog / bird).

Key Points

- Classification predicts categories; regression predicts numbers.
- Supervised learning requires labelled examples.
- We always evaluate a model on data it has *not* seen during training, because memorising the training data is easy — generalising to new data is the real goal.

0.1.4 The Running Dataset for the Worked Examples

To keep things concrete, the worked examples use a small, imaginary dataset about students applying to a coding bootcamp. Each row is one applicant, and we want to predict whether they will be **Admitted** (1) or **Not admitted** (0).

Table 2: The mini "bootcamp admissions" dataset used in worked examples.

Applicant	Hours studied	Practice test score	Admitted (y)
A	2	45	0
B	4	55	0
C	5	60	0
D	6	62	1
E	8	70	1
F	10	85	1

0.2 Logistic Regression

0.2.1 The Problem With Using a Straight Line

Suppose we try to predict admission (0 or 1) from hours studied using ordinary linear regression — fitting a straight line. The line might output values like -0.3 or 1.4. What does a probability of 140% mean? Nothing sensible. Straight lines are not confined to the range between 0 and 1, but probabilities must be. We need a way to take any number the model computes and gently *squash* it into the interval $[0, 1]$.

Intuition & Analogy

Think of a **dimmer switch** for a light. A regular on/off switch is too abrupt: below some point the light is fully off, above it fully on. A dimmer moves smoothly from completely off (0) to fully on (1), passing through every brightness in between. Logistic regression replaces the hard on/off decision with a smooth dimmer: as the evidence (e.g. hours studied) increases, the predicted probability of "Admitted" rises smoothly from near 0 to near 1.

0.2.2 Step 1: Start With a Linear Score

Logistic regression first computes a plain linear score, exactly like linear regression would:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b \tag{1}$$

where each x_i is a feature (hours studied, test score, ...), each w_i is a **weight** saying how important that feature is, and b is a **bias** (a baseline shift). A large positive z means "strong evidence for class 1"; a large negative z means "strong evidence for class 0"; $z = 0$ means the evidence is perfectly balanced.

0.2.3 Step 2: Squash the Score With the Sigmoid Function

The score z can be any number from $-\infty$ to $+\infty$. To turn it into a probability, we pass it through the **sigmoid** (also called *logistic*) function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

Let us understand it by plugging in a few values, no calculus needed:

- If $z = 0$: $\sigma(0) = \frac{1}{1+e^0} = \frac{1}{2} = 0.5$. Balanced evidence gives a 50/50 probability. Sensible.
- If $z = +4$: $e^{-4} \approx 0.018$, so $\sigma(4) \approx \frac{1}{1.018} \approx 0.98$. Strong positive evidence gives probability near 1.
- If $z = -4$: $e^4 \approx 54.6$, so $\sigma(-4) \approx \frac{1}{55.6} \approx 0.02$. Strong negative evidence gives probability near 0.

No matter how extreme z becomes, the output never escapes the interval $(0, 1)$. That is exactly what a probability requires.

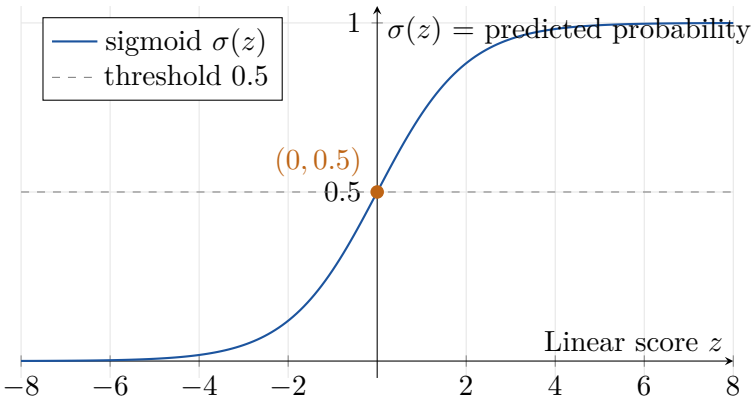

Figure 1: The sigmoid function smoothly maps any score z into a probability between 0 and 1. The horizontal dashed line marks the usual decision threshold of 0.5.

0.2.4 Step 3: Make a Decision With a Threshold

The model now outputs a probability, e.g. $P(\text{Admitted}) = 0.82$. To turn this into a final yes/no answer we compare against a **threshold**, usually 0.5:

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{if } \sigma(z) < 0.5 \end{cases} \tag{3}$$

Because $\sigma(z) = 0.5$ exactly when $z = 0$, the set of points where $z = 0$ forms the **decision boundary**: the invisible fence separating the region predicted as class 1 from the region predicted as class 0. With two features this boundary is a straight line; with three features, a flat plane. Logistic regression therefore draws *straight* boundaries — an important limitation to remember.

0.2.5 Step 4: How Does the Model Learn the Weights?

The model must choose weights w and bias b that fit the training data well. "Well" is measured by a **loss function**: a score that is small when predictions are good and large when they are bad. Logistic regression uses the **log loss** (also called *binary cross-entropy*):

$$\text{Loss} = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)\Big] \tag{4}$$

where $p_i = \sigma(z_i)$ is the predicted probability for example i and N is the number of training examples.

The formula looks intimidating, so read it as a story instead. For each example, only one of the two terms is active:

- If the true label is $y_i = 1$, the loss for that example is $-\log(p_i)$. If the model confidently said $p_i = 0.99$, then $-\log(0.99) \approx 0.01$: a tiny penalty. If the model confidently said $p_i = 0.01$ (confidently *wrong*), then $-\log(0.01) \approx 4.6$: a huge penalty.
- If the true label is $y_i = 0$, the mirror-image term $-\log(1 - p_i)$ plays the same role.

In short: *being confidently wrong is punished severely; being confidently right is rewarded.* The training algorithm (**gradient descent**) repeatedly nudges the weights in whatever direction reduces this loss, like a hiker walking downhill in small steps until reaching the bottom of a valley.

0.2.6 Interpreting the Weights: Odds and Log-Odds

A pleasant bonus of logistic regression is that its weights are interpretable. Rearranging the sigmoid gives:

$$\log\left(\frac{p}{1-p}\right) = z = w_1 x_1 + \cdots + w_n x_n + b \tag{5}$$

The quantity $\frac{p}{1-p}$ is called the **odds** (familiar from sports betting: a probability of 0.75 means odds of 3:1). The equation says: *increasing feature x_1 by one unit adds w_1 to the log-odds*, i.e. multiplies the odds by e^{w_1}. If the weight on "hours studied" is 0.7, each extra hour multiplies the odds of admission by $e^{0.7} \approx 2$ — doubling them.

Worked Example: Predicting Admission by Hand

Suppose training has produced the model

$$z = 0.9 \times (\text{hours}) - 5.4.$$

An applicant studied for 7 hours. Step by step:

1. Compute the score: $z = 0.9 \times 7 - 5.4 = 6.3 - 5.4 = 0.9$.
2. Apply the sigmoid: $\sigma(0.9) = \dfrac{1}{1+e^{-0.9}} = \dfrac{1}{1+0.4066} \approx 0.71$.
3. Compare with the threshold: $0.71 \geq 0.5$, so predict **Admitted (1)** with 71% confidence.

Now try an applicant with 4 hours: $z = 0.9 \times 4 - 5.4 = -1.8$, and $\sigma(-1.8) \approx 0.14$. Prediction: **Not admitted (0)**. Notice also that the decision boundary sits where $z = 0$, i.e. at hours $= 5.4/0.9 = 6$ hours: applicants studying more than 6 hours are predicted to be admitted.

Hands-On Python Notebook

The full Python implementation for this topic lives in the companion notebook `01_Logistic_Regression.ipynb`. It covers (i) fitting logistic regression on the mini bootcamp dataset with `StandardScaler` + `LogisticRegression` and reading the learned weights, and (ii) a complete real-data workflow on the Titanic dataset (`titanic_train.csv`): exploratory data analysis, missing-value imputation, dummy-variable encoding, train/test split, training, and evaluation.

0.2.7 Assumptions of Logistic Regression

- The relationship between the features and the *log-odds* of the outcome is linear (the decision boundary is a straight line/plane).
- Training examples are independent of one another.
- Features are not strongly correlated with each other (no severe *multicollinearity*), otherwise the weights become unstable and hard to interpret.
- There are no extreme outliers dominating the fit.

0.2.8 Advantages and Disadvantages

Table 3: Strengths and weaknesses of logistic regression.

Advantages	**Disadvantages**
Simple, fast to train, works well on small datasets	Can only learn straight-line (linear) decision boundaries
Outputs genuine probabilities, not just labels	Struggles when classes are separated by curved or complex shapes
Weights are interpretable (odds ratios)	Sensitive to strongly correlated features
Rarely overfits when features are few	Requires feature scaling for smooth training
A strong, honest baseline for any classification task	Needs manual feature engineering to capture interactions

Common Mistakes

- **Forgetting that "regression" is in the name only.** Logistic regression is a *classification* algorithm; the name survives for historical reasons.
- **Skipping feature scaling.** Unscaled features slow down or destabilise training.
- **Reading weights as probabilities.** A weight of 0.7 is a change in *log-odds*, not a probability.
- **Always using the 0.5 threshold.** In medical screening you may prefer 0.3 to catch more true cases; the threshold is a dial you are allowed to turn.
- **Using accuracy alone on imbalanced data** (covered in detail in Topic 4: Classification Metrics).

Key Points

- Logistic regression = linear score z + sigmoid squashing + threshold.
- The sigmoid converts any score into a probability in $(0, 1)$.
- Log loss punishes confident wrong answers heavily.
- The decision boundary is where $z = 0$, and it is always linear.
- Weights translate into odds ratios via e^w.